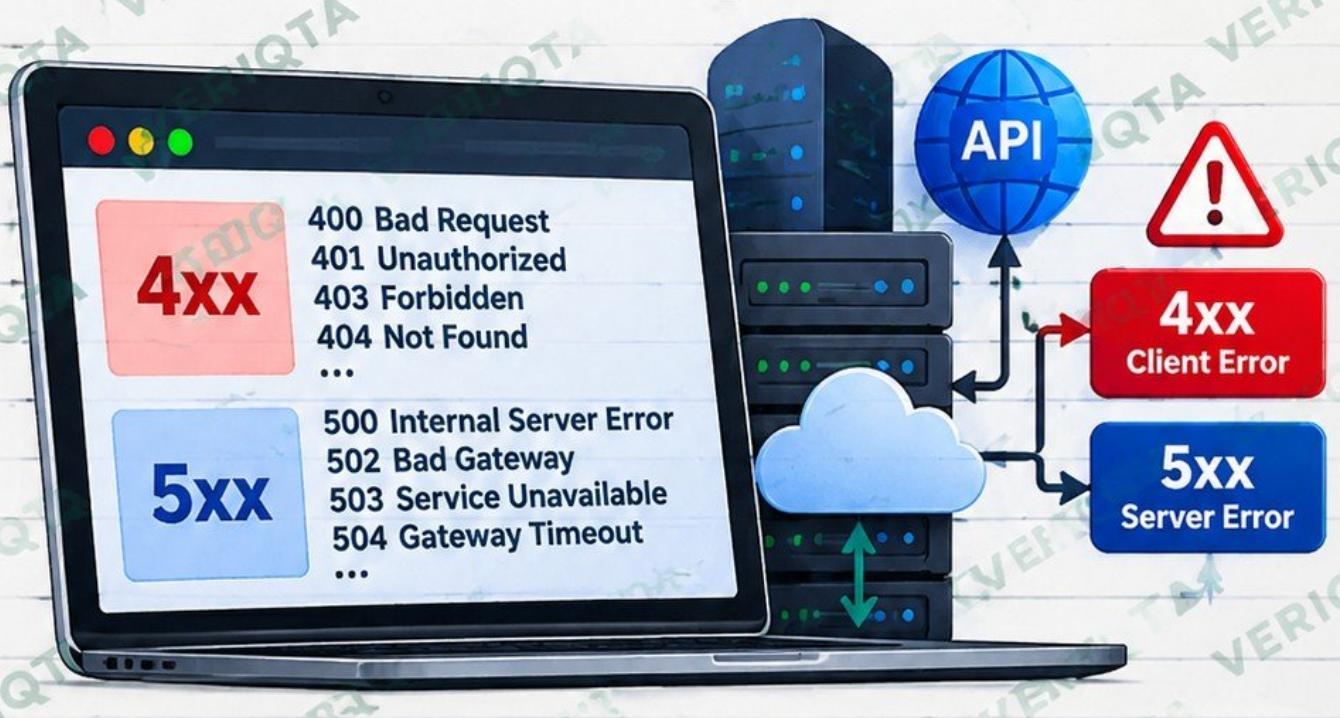
VERIQTA

HTTP ERROR CODES HANDBOOK

40 Real 4xx and 5xx Troubleshooting Scenarios



@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

LEARN | PRACTICE | TROUBLESHOOT | BUILD

VERIQTA

HTTP 400: Bad Requests in Production

01 An API rejects a request with HTTP 400

Incident

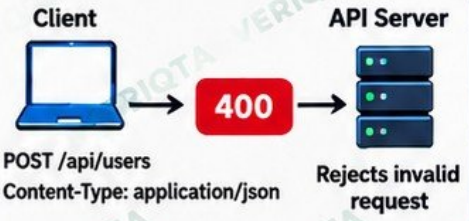
A frontend submits JSON to an API, but the server responds with HTTP 400 Bad Request.

What it means

- The server cannot understand or process the request because it is invalid or malformed.
- This is a client-side error (4xx).

Common causes

- Malformed JSON (syntax errors)
- Invalid or missing required fields
- Incorrect field types (e.g. string instead of number)
- Missing or wrong Content-Type
- Request validation rules failed



How to investigate

1. Reproduce the request with curl:

```
curl -i -X POST https://api.example.com/users
  -H "Content-Type: application/json"
  -d '{"name": "John"}'
```

2. Check the request body for JSON syntax errors.
3. Verify Content-Type header is correct.
4. Check application logs for validation errors.

Example error response

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
{
  "error": "Invalid request",
  "message": "'age' must be a number"
}
```

Fix and verify

- Correct the request payload.
- Use the right Content-Type.
- Ensure all required fields and types are valid.
- Test again and confirm the API returns 200.

02 Requests fail with 400 after passing through Nginx

Incident

A request returns HTTP 400 after passing through Nginx before reaching the application.

What it means

- Nginx (or a proxy) detected an invalid request and rejected it.
- It may be a malformed request, invalid headers, or oversized headers.

Common causes

- Malformed request syntax
- Invalid or illegal headers
- Oversized headers (e.g. large cookies)
- Invalid characters in the URL
- Request size exceeding limits
- Nginx rejecting the request before it reaches the application



How to investigate

1. Test the request directly:

```
curl -i https://your-domain.com/api/endpoint
```

2. Check Nginx error logs:

```
sudo tail -f /var/log/nginx/error.log
```

3. Compare with direct access (bypass Nginx):

```
curl -i http://localhost:8080/api/endpoint
```

4. Look for specific errors in the Nginx log (e.g. invalid header, request header too large, malformed request line).

Example Nginx error log

```
client sent invalid header while reading client request
or
client sent too long header line
or
invalid request line in client request
```

Fix and verify

- Fix the request format, headers, or URL.
- If headers are too large, review cookie size or increase Nginx limits (e.g. large_client_header_buffers).
- Test again and confirm the request reaches the application and returns 200.

Engineering focus Use curl -i, application logs, and Nginx error logs to identify which layer rejected the request.

VERIQTA

HTTP 401: Authentication Failures

03 An API returns 401 despite a supplied token

Incident

An API returns HTTP 401 Unauthorized even though a token is supplied in the request.

What it means

- The request is missing valid authentication credentials or the provided credentials are not accepted by the server.
- This is an authentication error (401), not an authorization error (403).

Common causes

- Expired token
- Invalid token signature
- Incorrect audience (aud)
- Issuer (iss) mismatch
- Malformed Authorization header
- Wrong token type (e.g. using access token where ID token is required)
- Revoked token
- Clock time difference between client and server

How it works



How to investigate

1. Check the request with curl (include the token):
```
curl -i -H "Authorication: Bearer <token>" https://api.example.com/users
```
2. Verify the token (decode and check claims):
```
echo "<token>" | jq -R 'split(".") | .[1] | @base64d | fromjson"
```
3. Check token details: expiry (exp), issuer (iss), audience (aud).
4. Check application logs for authentication errors.

Example response

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json
WWW-Authenticate: Bearer error="invalid_token",
                  error_description="Token is expired"
{
  "error": "invalid_token",
  "message": "The token has expired"
}
```

Fix and verify

- Obtain a valid token and ensure it is not expired.
- Use the correct token type and audience.
- Ensure the Authorication header format is correct: `Authorication: Bearer <token>`
- Verify server and client time are synchronized.
- Test again and confirm the API returns 200.

04 A Kubernetes application suddenly returns 401 after deployment

Incident

A Kubernetes application that worked before now returns HTTP 401 Unauthorized after a new deployment.

What it means

- The application is not receiving valid authentication credentials or is unable to validate the token.
- This can be due to missing configuration, incorrect Secrets, or changed token validation settings.

Common causes

- Missing authentication configuration
- Incorrect Kubernates Secret reference
- Wrong environment variables
- Invalid token signing key or issuer
- Changed token validation settings
- Service account token issues
- Ingress or proxy removing headers
- Application not loading configuration at startup
- Using the wrong namespace or Secret

Common flow in Kubernetes



How to investigate

1. Check the application logs:
```
kubectl logs -n <namespace> <pod-name>
```
2. Verify environment variables and configuration:
```
kubectl exec -n <namespace> <pod-name> -- printenv | grep -i token
```
3. Check the Secret and its values:
```
kubectl get secret -n <namespace> <secret-name> -o yaml
```
4. Test the endpoint directly from inside the cluster:
```
kubectl exec -n <namespace> <pod-name> -- curl -i http://localhost:8080/api
```

Example error log

```
Authentication failed: invalid token signature
or
JWT verification failed: issuer mismantch
or
No valid credentials provided
```

Fix and verify

- Ensure the correct Secret is referenced in the deployment.
- Verify environment variables and token validation settings.
- Confirm the application is using the correct issuer and audience.
- Redeploy or restart the application.
- Test again and confirm the endpoint returns 200.

Engineering focus Distinguish missing or invalid authentication from insufficient authorization. Never expose live tokens in logs or screenshots.

VERIQTA

HTTP 403: Access Denied

05 A user authenticates successfully but receives 403

Incident

A user logs in and authenticates successfully, but the API returns HTTP 403 Forbidden when accessing a specific resource.

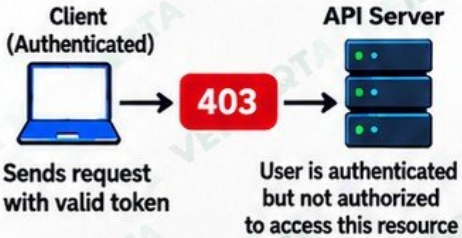
What it means

- The request is understood and the user is authenticated, but the user does not have permission to access the resource.
- This is an authorization error (403), not an authentication error (401).

Common causes

- User lacks required role or permission
- Missing API scope
- RBAC policy does not allow access
- Resource belongs to another user or organization
- API expects a different role (e.g. admin)
- Incorrect group membership
- Policy or IAM rules denying access

How it works



How to investigate

1. Verify the token and user identity:
```
curl -i -H "Authorication: Bearer <token>" \
https://api.example.com/resource
```
2. Check the required permissions or scope for the endpoint.
3. Inspect application logs for authorization details.
```
kubectl logs <pod-name> | grep -i 403
```
4. Review RBAC or IAM policies (e.g. Kubernetes RBAC, AWS IAM).

Example response

```
HTTP/1.1 403 Forbidden
Content-Type: application/json
{
  "error": "forbidden",
  "message": "You do not have permission
  to access this resource"
}
```

Fix and verify

- Assign the correct role or permission.
- Add the required API scope.
- Update RBAC, IAM, or application policy.
- Test again and confirm the request returns 200.

06 A website returns 403 after a security configuration change

Incident

A website that was working previously now returns HTTP 403 Forbidden after a security configuration change.

What it means

- The web server or firewall is blocking access to the requested resource.
- This can be due to access rules, IP restrictions, filesystem permissions, or security policies.

Common causes

- Nginx access rules (allow/deny)
- Blocked client IP address
- Web Application Firewall (WAF) blocking the request
- Missing or incorrect filesystem permissions
- Hotlinking or referer restrictions
- IP or country blocking rules
- Security module (e.g. ModSecurity) blocking the request

How it works (Nginx example)



Example Nginx error log

```
2026/10/27 12:14:33 [error] 12345#12345:
*5678 access forbidden by rule,
client: 203.0.113.10,
server: your-domain.com,
request: "GET /admin HTTP/1.1",
host: "your-domain.com"
```

How to investigate

1. Test the request and check response headers:
```
curl -i https://your-domain.com/
```
2. Check Nginx access and error logs:
```
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log
```
3. Check firewall or WAF logs (e.g. ModSecurity, Cloudflare, AWS WAF).
4. Verify filesystem permissions if it is a static file:
```
ls -l /var/www/html/ && sudo chmod -R 755 /var/www/html/
```

Fix and verify

- Update Nginx allow/deny rules or WAF policy.
- Unblock the correct IP address (if legitimate).
- Correct filesystem permissions.
- Test again from the client and confirm the site returns 200.

Engineering focus Identify the component enforcing access restrictions before changing permissions.

VERIQTA

HTTP 404: Resources That Cannot Be Found

07 An API endpoint returns 404 after deployment

Incident

An API endpoint returns HTTP 404 Not Found after a new deployment, even though the application is running.

What it means

- The server cannot find the requested resource at the specified URL.
- This can happen when the path is incorrect, the route is not registered, or the API version/prefix has changed.
- This is a client-side error (4xx), not a server failure (5xx).

Common causes

- Route not registered in the application
- API version changed (e.g. /v1 to /v2)
- Missing or incorrect URL prefix
- Typo in the endpoint path
- Endpoint disabled or removed
- Proxy or load balancer not forwarding the correct path
- Case-sensitive path mismatch

Example request and response

```
curl -i https://api.example.com/users
```

```
HTTP/1.1 404 Not Found
Content-Type: application/json

{
  "error": "not_found",
  "message": "The requested resource
   was not found"
}
```

How to investigate

1. Check the exact URL and method:
   ```
   curl -i https://api.example.com/users
   ```
2. Check available routes (if possible):
   ```
   curl -i https://api.example.com/
   ```
3. Check application logs for route information.
4. Verify API version and prefix in documentation or configuration.

Fix and verify

- Use the correct endpoint path and method.
- Register the route in the application if it is missing.
- Update client configuration if the API version changed.
- Test again and confirm the API returns 200.

Example structure

✓ Correct
```
GET /api/v1/users
```

✗ Incorrect (wrong version)
```
GET /api/users
```

✗ Incorrect (missing prefix)
```
GET /v1/users
```

✗ Incorrect (typo)
```
GET /api/v1/user
```

08 Kubernetes Ingress returns 404 while application Pods are healthy

Incident

A Kubernetes Ingress returns HTTP 404 Not Found, but the application Pods are healthy and running.

What it means

- The Ingress controller cannot find a matching rule for the requested host or path.
- The request does not reach the application Pods.
- This is usually a routing or configuration issue, not a Pod failure.

Common causes

- Hostname rule does not match
- Path rule does not match
- Incorrect pathType (Prefix vs Exact)
- Missing or incorrect Service backend configuration
- Default backend returning 404
- Ingress controller configuration issues
- DNS pointing to the wrong Ingress
- Multiple Ingress resources with conflicting rules

How it works



Example Ingress rule (YAML)

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  rules:
  - host: api.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: app-service
            port:
              number: 8080
```

Fix and verify

- Update the Ingress hostname or path rules.
- Use the correct pathType (Prefix or Exact).
- Ensure the Service points to the correct port and Pods.
- Check DNS if the wrong Ingress is receiving traffic.
- Test again and confirm the request returns 200 from the application.

How to investigate

1. Check Ingress rules:
   ```
   kubectl get ingress -n <namespace> -o yaml
   ```
2. Verify that the hostname and path match your request.
3. Check the Service and endpoints:
   ```
   kubectl get svc -n <namespace>
   kubectl get endpoints -n <namespace>
   ```
4. Test directly from inside the cluster (bypass Ingress):
   ```
   kubectl run -it --rm test --image=curlimages/curl -- sh
   curl -i http://<service-name>:<port//<path>
   ```

Engineering focus Trace the request from hostname and path through the ingress controller to the application.

VERIQTA

HTTP 405 and 406: Method and Response Negotiation Failures

09 A deployment API returns 405 Method Not Allowed

Incident

A deployment API returns HTTP 405 Method Not Allowed when you send a POST request to the endpoint.

What it means

- The server understands the request, but the HTTP method is not allowed for this resource.
- The endpoint exists, but it only allows specific methods (e.g. GET).
- This is a client-side error (4xx), not a server failure.

Common causes

- Using the wrong HTTP method (e.g. POST instead of GET)
- Endpoint only supports specific methods
- Route method configuration in the application
- Proxy or API gateway restrictions
- API documentation not followed
- Using a method not implemented in the server

Example request and response

```
curl -i -X POST https://api.example.com/deploy

HTTP/1.1 405 Method Not Allowed
Allow: GET, PUT
Content-Type: application/json

{
  "error": "method_not_allowed",
  "message": "POST is not allowed.
   Allowed methods: GET, PUT"
}
```

How to investigate

1. Check the allowed methods:
   ```
   curl -i -X OPTIONS https://api.example.com/deploy
   ```
2. Review the API documentation.
3. Check application or gateway logs.
4. Verify the URL and method are correct.

Fix and verify

- Use the correct HTTP method.
- Update client code or configuration.
- If the endpoint should support the method, update the application route configuration.
- Test again and confirm a successful response (200).

Example: Correct request

```
curl -i -X GET
https://api.example.com/deploy

HTTP/1.1 200 OK
Content-Type: application/json

{
  "status": "success",
  "deployments": []
}
```

10 An API returns 406 Not Acceptable

Incident

An API returns HTTP 406 Not Acceptable when you send a request with a specific Accept header.

What it means

- The server understood the request, but it cannot produce a response in the requested format.
- This happens due to content negotiation rules.
- The resource exists, but the server does not support the requested response type.

Common causes

- Accept header requests a format not supported by the server
- Server only supports specific response formats (e.g. JSON or XML)
- Incorrect or unsupported media type in Accept header
- API version requires a specific format
- Proxy or gateway modifying headers
- Misconfigured content negotiation in the application

Example request and response

```
curl -i -H "Accept: application/xml" \
  https://api.example.com/users

HTTP/1.1 406 Not Acceptable
Content-Type: application/json

{
  "error": "not_acceptable",
  "message": "The requested response
   format is not supported"
}
```

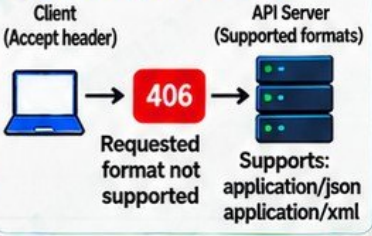
How to investigate

1. Check the Accept header you are sending:
   ```
   curl -i -H "Accept: application/xml" https://api.example.com/users
   ```
2. Try a supported format (e.g. application/json):
   ```
   curl -i -H "Accept: application/json" https://api.example.com/users
   ```
3. Check API documentation for supported response formats.
4. Review application logs for content negotiation details.

Fix and verify

- Use a supported Accept header (e.g. application/json).
- Update the client to handle the correct response format.
- If the server should support additional formats, update the application configuration.
- Test again and confirm a 200 response.

Content negotiation flow



Engineering focus Understand the difference between the HTTP method, request content type, and acceptable response type.

VERIQTA

HTTP 408 and 409: Timeouts and State Conflicts

11 Nginx returns 408 while waiting for a client request

Incident

Nginx returns HTTP 408 Request Timeout while waiting for a client request.

What it means

- Nginx did not receive a complete request from the client within the configured timeout period.
- This happens before the request reaches the application.
- It is a client-side issue (4xx), not a server failure.

Common causes

- Slow client sending the request
- Incomplete request headers or body
- Large request body sent slowly
- Network interruptions
- client_header_timeout or client_body_timeout too low
- Proxy or load balancer closing the connection
- High network latency

How it works



Client starts sending request but does not complete it in time

Nginx waits for the full request (headers and/or body) and returns 408

How to investigate

1. Check the Nginx error log:
   ```
   sudo tail -f /var/log/nginx/error.log
   ```
2. Check Nginx timeout settings:
   ```
   grep -E 'client_header_timeout|client_body_timeout' \
   /etc/nginx/nginx.conf
   ```
3. Inspect client network and request behavior:
   ```
   curl -v --limit-rate 1k https://your-domain.com/ api
   ```
4. Check for proxy or load balancer timeouts (if applicable).

Example Nginx error log

```
client timed out while
reading client request
headers, client: 203.0.113.10,
server: your-domain.com,
request: "POST /api/upload
HTTP/1.1", host: "your-domain.com"
```

Fix and verify

- Increase client_header_timeout or client_body_timeout (if appropriate).
- Ensure clients send complete requests within the timeout.
- Check network stability and remove interruptions.
- Review proxy or load balancer timeout settings.
- Test again and confirm the request reaches the application.

12 An API returns 409 when two processes update the same resource

Incident

An API returns HTTP 409 Conflict when two processes try to update the same resource at the same time.

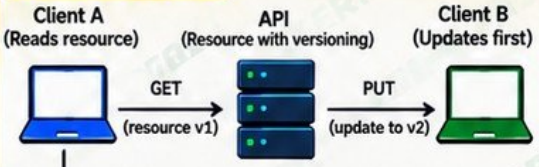
What it means

- The request is valid, but it conflicts with the current state of the resource.
- This happens due to concurrent updates or version mismatches.
- It is a client-side error (4xx), not a server failure.

Common causes

- Two clients updating the same resource at the same time
- Optimistic concurrency control
- Mismatched resource version or ETag
- Stale data in the client request
- Database or application concurrency checks
- Application intentionally rejecting the update to prevent data loss

How it works



Client A tries to update using stale data (v1) → 409 API rejects the update due to version conflict

How to investigate

1. Check the API response and error message:
   ```
   curl -i -X PUT https://api.example.com/resource \
   -H "Content-Type: application/json" -d '{...}'
   ```
2. Look for versioning fields (ETag, resourceVersion, etc.).
3. Check application logs for conflict details.
4. Review the current state of the resource before retrying:
   ```
   curl -s https://api.example.com/resource | jq
   ```

Example response

```
HTTP/1.1 409 Conflict
Content-Type: application/json
{
  "error": "conflict",
  "message": "Resource has been
   modified by another process",
  "currentVersion": "3",
  "yourVersion": "2"
}
```

Fix and verify

- Retrieve the latest resource state before updating.
- Use the correct version, ETag, or resourceVersion.
- Implement retry logic with conflict resolution.
- Consider optimistic locking or merge strategies.
- Test again and confirm the update succeeds.

Engineering focus Distinguish an incomplete client request from a concurrency conflict. Understand why retrying a 409 requires reviewing the current resource state.

VERIQTA

HTTP 410 and 412: Removed Resources and Failed Preconditions

13 An old API endpoint returns 410 Gone

Incident

An old API endpoint returns HTTP 410 Gone because the resource has been intentionally removed or retired.

What it means

- The requested resource used to exist but is permanently removed.
- The server is intentionally indicating that the resource is no longer available and will not be restored.
- This is a client-side error (4xx), not a server failure.

Common causes

- Endpoint or API version has been deprecated and removed
- Resource URL no longer valid
- Service migrated to a new API version or path
- Documentation not updated
- Old client integrations still using retired endpoints
- API gateway or reverse proxy explicitly returning 410

Example request and response

```
curl -i https://api.example.com/v1/users

HTTP/1.1 410 Gone
Content-Type: application/json
{
   "error": "gone",
   "message": "This endpoint has been
   permanently removed.",
} "info": "Use /v2/users instead"
```

How to investigate

1. Check the URL and API version:
   `curl -i https://api.example.com/v1/users`
2. Review API documentation or change logs for deprecated endpoints.
3. Check application logs for 410 responses.
4. Confirm the new endpoint or migration path.

Fix and verify

- Update the client to use the new endpoint or API version.
- Remove references to the old endpoint from code and configuration.
- Test the new endpoint and confirm it returns 200.
- Update internal documentation and inform dependent teams.

Example migration

Old (retired)	New (active)
/v1/users →	/v2/users

- Update client code, scripts, and integrations.
- Verify the new endpoint returns the expected data.
- Monitor for any remaining 410 requests in logs.

14 A resource update fails with 412 Precondition Failed

Incident

A resource update fails with HTTP 412 Precondition Failed when the provided precondition (e.g. ETag or If-Match) does not match the current resource state.

What it means

- The server expected a specific condition to be true, but it was not.
- This usually happens when the resource has been modified since it was last retrieved.
- It is a client-side error (4xx), not a server failure.

Common causes

- If-Match header with a stale ETag
- Resource version has changed on the server
- Concurrent updates by another client
- Missing or incorrect ETag value
- Optimistic concurrency control is enabled
- Client using outdated resource version for the update

Example request and response

```
curl -i -X PUT https://api.example.com/users/123 \
  -H "If-Match: \"abc123\"" \
  -H "Content-Type: application/json" \
  -d '{"name":"New Name"}'

HTTP/1.1 412 Precondition Failed
Content-Type: application/json
{
  "error": "precondition_failed",
  "message": "The resource has been modified
   by another process.",
  "currentEtag": "def456"
}
```

How to investigate

1. Check the current ETag or resource version:
   `curl -i https://api.example.com/users/123`
2. Compare the ETag with the one used in If-Match.
3. Review application logs to identify conflicting updates.
4. Check if multiple clients or processes are updating the same resource.

Example workflow

1. Get the latest resource
   `GET /users/123`
   `ETag: "def456"`
   ↓
2. Update with correct ETag
   `PUT /users/123`
   `If-Match: "def456"`
   ↓
3. Success
   `HTTP/1.1 200 OK`

Fix and verify

- Retrieve the latest resource state before making the update.
- Use the correct and current ETag or resource version.
- Implement retry logic to handle version conflicts (e.g. fetch latest, merge changes, retry).
- Ensure the client handles 412 responses gracefully.
- Test again and confirm the update succeeds with HTTP 200.

Engineering focus Understand how APIs protect resources from outdated writes and communicate permanent resource removal.

VERIQTA

HTTP 413 and 414: Request Size and URL Limits

15 A file upload returns 413 Content Too Large

Incident

A file upload returns HTTP 413 Content Too Large when trying to upload a file to the server.

What it means

- The request is valid, but the request body (file size) exceeds the allowed limit.
- The server, proxy, or application has a configured maximum upload size.
- It is a client-side error (4xx), not a server failure.

Common causes

- Nginx client_max_body_size is too small
- Kubernetes Ingress body size limit (e.g. ingress controller)
- Application upload limit (e.g. Spring, Express, PHP, etc.)
- Reverse proxy or load balancer has a request size limit
- Cloud provider or WAF limiting the request size
- Large file uploads without chunking

Example request and response

```
curl -i -F "file=@largefile.zip" \
   https://api.example.com/upload

HTTP/1.1 413 Content Too Large
Content-Type: text/html

<html>
 <head><title>413 Request Entity
 Too Large</title></head>
 <body>
 <h1>413 Request Entity Too Large</h1>
 </body>
</html>
```

How to investigate

1. Check the file size:
   ```
   ls -lh largefile.zip
   ```
2. Check Nginx configuration:
   ```
   grep client_max_body_size /etc/nginx/nginx.conf
   ```
3. Check Ingress limits (Kubernetes):
   ```
   kubectl get ingress -n <namespace> -o yaml
   ```
4. Check application upload settings and logs.

Fix and verify

- Increase client_max_body_size in Nginx (if applicable).
- Update Ingress body size limit annotations.
- Adjust application upload limit (e.g. Spring, Express, PHP).
- Test again with a file within the new limit and then with the expected file size.
- Verify the upload succeeds and is processed correctly.

Example Nginx configuration

```
http {
    client_max_body_size 50m;
}

server {
    listen 80;
    server_name api.example.com;
    location /upload {
       client_max_body_size 50m;
       proxy_pass http://backend;
    }
}
```

16 A request returns 414 URI Too Long

Incident

A request returns HTTP 414 URI Too Long when the URL is too long.

What it means

- The requested URL (including query string) exceeds the allowed length.
- The server, proxy, or client has a limit on the maximum URL size.
- It is a client-side error (4xx), not a server failure.

Common causes

- Excessively long query strings (e.g. many parameters or large encoded values)
- Using GET requests for large payloads
- URL length limits in Nginx, Apache, IIS or load balancers
- Redirects that append large parameters
- Client or browser URL length limitations

Example request and response

```
curl -i "https://api.example.com/search?query=
$(python -c 'print("a"*5000)')"

HTTP/1.1 414 URI Too Long
Content-Type: text/html

<html>
 <head><title>414 Request-URI Too Long</title></head>
 <body>
   <h1>414 Request-URI Too Long</h1>
 </body>
</html>
```

How to investigate

1. Check the full URL length (number of characters).
   ```
   echo -n "https://api.example.com/?$(python -c 'print("a"*5000)')" | wc -c
   ```
2. Look at server or proxy logs (Nginx/Apache) for 414 errors.
   ```
   sudo tail -f /var/log/nginx/error.log
   ```
3. Identify which layer is returning the 414 (client, proxy, or application).
4. Review API design and consider using POST for large data.

Fix and verify

- Use POST requests for large payloads instead of GET.
- Reduce the size of query parameters.
- Adjust server/proxy URL length limits (if appropriate).
- Test again and confirm the request succeeds.

Better approach (use POST)

✓ POST /search
Content-Type: application/json

```
{
  "query": "large search data here",
  "filters": [...]
}
```

✗ Avoid very long URLs with large query strings.

Engineering focus Determine which layer enforces request limits before modifying configurations.

VERIQTA

HTTP 415 and 422: Unsupported Content and Validation Failures

17 An API returns 415 Unsupported Media Type

Incident

An API returns HTTP 415 Unsupported Media Type when the request is sent with an unsupported content type or format.

What it means

- The server understands the request, but the media type of the request is not supported for this endpoint.
- This happens when the Content-Type is missing, incorrect, or the format is not supported (e.g. JSON sent as plain text).
- It is a client-side error (4xx), not a server failure.

Common causes

- Missing or incorrect Content-Type header
- Unsupported media type (e.g. sending XML to a JSON-only endpoint)
- JSON data sent as plain text
- Incorrect request format (e.g. multipart/form-data instead of application/json)
- Endpoint expects a specific media type (e.g. application/json)
- Proxy or API gateway modifying or removing the Content-Type header

Example request and response

```
curl -i -X POST https://api.example.com/users\
  -H "Content-Type: text/plain" \\
  -d '{"name": "John"}'
```

```
HTTP/1.1 415 Unsupported Media Type
Content-Type: application/json

{
  "error": "unsupported_media_type",
  "message": "Content-Type 'text/plain'
  is not supported. Use 'application/json'."
}
```

How to investigate

1. Check the Content-Type header:
   ```
   curl -i -X POST https://api.example.com/users \\
    -H "Content-Type: application/json"
   ```
2. Verify the request format (JSON, XML, form-data, etc.).
3. Check the API documentation for supported media types.
4. Review application logs to see the expected content type.

Fix and verify

- Use the correct Content-Type header (e.g. application/json).
- Send the request in the supported format.
- Update client code or configuration to use the correct media type.
- Test again and confirm the API returns 200.

Example: Correct request

```
curl -i -X POST https://api.example.com/users\
  -H "Content-Type: application/json" \
  -d '{"name": "John"}'
```

```
HTTP/1.1 201 Created
Content-Type: application/json
{
   "id": 123,
   "name": "John"
}
```

18 An API returns 422 Unprocessable Content

Incident

An API returns HTTP 422 Unprocessable Content when the request is syntactically valid but fails business validation rules.

What it means

- The server understands the content type and the request format, but it cannot process the instructions due to invalid data or business validation errors.
- This is a client-side error (4xx), not a server failure.

Common causes

- Missing required fields
- Invalid field values (e.g. wrong format, out of range)
- Business validation rules (e.g. duplicate values, forbidden combinations)
- Referencing non-existent resources
- Data that violates application constraints

Example request and response

```
curl -i -X POST https://api.example.com/users
  -H "Content-Type: application/json"
  -d '{ "name": "", "email": "invalid-email" }'
```

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/json
{
   "error": "validation_failed",
   "message": "The request data is invalid.",
   "errors": {
      "name": ["Name is required"],
      "email": ["Email format is invalid"]
   }
}
```

How to investigate

1. Check the request payload for missing or invalid fields.
2. Verify the data against API validation rules.
3. Review the API response for detailed validation errors.
4. Check application logs for additional validation messages.

Fix and verify

- Provide all required fields with valid values.
- Ensure the data meets business validation rules.
- Update client code or input data.
- Test again and confirm the API returns 200.

Example: Correct request

```
curl -i -X POST https://api.example.com/users \\
  -H "Content-Type: application/json" \
  -d '{ "name": "John", "email": "john@example.com" }'
```

```
HTTP/1.1 201 Created
Content-Type: application/json
{  "id": 123,
   "name": "John",
}  "email": "john@example.com"
```

Engineering focus Differentiate malformed JSON, unsupported media types, and semantically invalid request data.

VERIQTA

HTTP 429 and 431: Rate Limits and Oversized Headers

19 An API returns 429 during a traffic spike

Incident

An API returns HTTP 429 Too Many Requests during a traffic spike when the client sends too many requests in a short period.

What it means

- The server is rate limiting requests because the client has exceeded the allowed request rate.
- This is usually a protection mechanism to prevent abuse and maintain service stability.
- It is a client-side error (4xx), not a server failure.

Common causes

- Client sending too many requests in a short time (e.g. traffic spike or load test)
- Rate limiting rules at API gateway, load balancer or application
- Retry storms from multiple clients
- Bots or automated scripts making excessive requests
- Shared IP or API key hitting a global limit
- Missing or incorrect rate limit configuration
- Burst traffic without backoff or throttling

Example request and response

```
curl -i https://api.example.com/data
```

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 60
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1700000000

{
  "error": "rate_limited",
  "message": "Too many requests.",
  "retry_after": 60
}
```

How to investigate

1. Check the response headers for rate limit details:
   ```
   curl -i https://api.example.com/data
   ```
2. Check API gateway or load balancer rate limit settings.
3. Review application logs for rate limit events.
4. Identify if there are retry storms or unusual traffic spikes.

Fix and verify

- Implement client-side rate limiting and backoff (e.g. exponential backoff).
- Respect the Retry-After header.
- Adjust API gateway or application rate limit configuration (if appropriate).
- Reduce unnecessary requests and optimize client behavior.
- Test again and confirm the requests succeed.

Example: Bounded retry with backoff

```
for i in {1..5}; do
  response=$(curl -s -o /dev/null
   -w "%{http_code} %{time_total} \n"
   https://api.example.com/data)
  if [ "$(echo $response | cut -d' ' -f1)"
   = "429" ]; then
    sleep $((2**i))  # exponential backoff
   else
     break
  fi
done
```

20 A browser receives 431 Request Header Fields Too Large

Incident

A browser receives HTTP 431 Request Header Fields Too Large when the request headers exceed the server's allowed size limit.

What it means

- The server is rejecting the request because the request headers are too large.
- This usually happens due to oversized cookies, large authentication headers, or excessive custom headers.
- It is a client-side error (4xx), not a server failure.

Common causes

- Very large cookies (e.g. session data, tracking cookies)
- Large authentication headers (e.g. JWT tokens)
- Too many custom headers
- Proxy or web server header size limits (e.g. Nginx, Apache, load balancer)
- Headers added by browser extensions or third-party tools
- Misconfigured proxy forwarding all client headers

Example request and response

```
curl -i https://www.example.com
```

```
HTTP/1.1 431 Request Header Fields Too Large
Content-Type: text/html

<html>
  <head><title>431 Request Header Fields
   Too Large</title></head>
  <body>
   <h1>431 Request Header Fields Too Large</h1>
   <p>The request headers are too large.</p>
  </body>
</html>
```

How to investigate

1. Check the size of request headers (especially cookies):
   ```
   curl -I https://www.example.com
   curl -H "Cookie: <your-cookie>" -I https://www.example.com
   ```
2. Identify large cookies or authentication tokens.
3. Review proxy or web server header size limits.
4. Check for unnecessary or duplicated headers.

Fix and verify

- Reduce cookie size or remove unnecessary cookies.
- Trim large authentication tokens if possible.
- Adjust proxy or web server header size limits (if appropriate).
- Remove unnecessary headers.
- Test again and confirm the request succeeds.

Example: Check header size

```
curl -I https://www.example.com

# Check cookie size
echo -n "<your-cookie-value>" | wc -c

# Check all request headers size
(approximate)
curl -v https://www.example.com 2>&1 | \
 grep "<" | awk '{print length, $0}' | sort -nr | head )
```

Engineering focus Apply bounded retries with backoff for rate limits and investigate header growth without exposing sensitive values.

VERIQTA

HTTP 500: Application Failures

21 An API returns 500 after a new deployment

Incident

An API returns HTTP 500 Internal Server Error affter a new deployment.

What it means

- The server encountered an unexpected condition and could not process the request.
- This usually happens due to application errors, misconfiguration, or failed dependencies.
- It is a server-side error (5xx), not a client failure.

Common causes

- Unhandled exceptions in the application code
- Missing or incorrect environment variables
- Application initialization failures
- Failed connection to database, cache or external services
- Incorrect configuration files (e.g. application.yaml, .env)
- Incompatible library or dependency versions
- File permission issues
- Failed migrations during startup

Example error response

```
HTTP/1.1 500 Internal Server Error
Content-Type: application/json

{
  "error": "internal_server_error",
  "message": "An unexpected error
   occurred.",
  "request_id": "a1b2c3d4"
}
```

How to investigate

1. Check application logs for error details:
   `kubectl logs <pod-name> | tail -n 100`
2. Check recent deployment changes:
   `kubectl rollout history deployment/<name>`
3. Verify environment variables and configuration:
   `kubectl get deployment <name> -o yaml`
4. Check application startup logs for initialization errors.
5. Verify dependencies (database, cache, external APIs).

Fix and verify

- Identify and fix the root cause from logs.
- Set correct environment variables and configuration.
- Ensure all dependencies are available and reachable.
- Verify database migrations completed successfully.
- Redeploy the application and monitor logs.
- Test again and confirm the API returns 200.

Example log (application)

```
2026-10-27 10:15:23 ERROR
[main] o.s.b.SpringApplication
: Application failed to start

Caused by: java.lang.IllegalState
Exception: Missing environment
variable: DATABASE_URL
    at com.example.config.Database
   Config.load(DatabaseConfig.java:42)
    ... 10 more
```

22 An application returns intermittent 500 errors under load

Incident

An application returns intermittent HTTP 500 errors when traffic is high or under load.

What it means

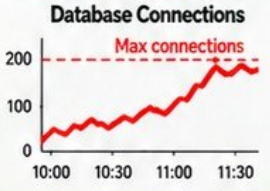
- The server is sometimes unable to process requests due to resource exhaustion or failing downstream operations.
- It may work under normal load but fail when resource limits are reached.
- It is a server-side error (5xx), not a client failure.

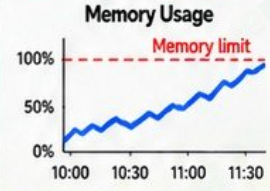
Common causes

- Database connection pool exhaustion
- Memory pressure (high memory usage or out of memory)
- Thread or worker pool starvation
- Slow or failing downstream services (APIs, queues, cache)
- Timeouts or circuit breaker triggered
- Resource limits (CPU, memory) reached in containers
- Inefficient queries or high latency dependencies

Example under load (logs and metrics)

```
2026-10-27 11:05:12 ERROR
com.zaxxer.hikari.pool.HikariPool
: Connection is not available, request timed out
  after 30000ms.
```





How to investigate

1. Check application logs and find the error pattern.
   `kubectl logs -f <pod-name>`
2. Correlate errors with metrics (CPU, memory, connections, latency).
3. Check database connection pool and database logs.
4. Identify failing downstream services or timeouts.
5. Review recent deployment changes and compare with previous version.

Fix and verify

- Increase resource limits if needed (CPU, memory, connection pool size).
- Optimize slow queries and external API calls.
- Implement proper timeouts, retry with backoff and circuit breakers.
- Scale the application if required.
- Monitor metrics and confirm the 500 errors are resolved.

Example database log

```
2026-10-27 11:05:12 WARN
org.postgresql.Driver
: FATAL: sorry, too many clients
already
2026-10-27 11:05:12 ERROR
Could not obtain JDBC Connection;
nested exception is
org.postgresql.util.PSQLException:
FATAL: remaining connection slots
are reserved for superuser connections
```

Engineering focus Correlate error logs, request IDs, application metrics, and deployment timestamps before identifying a root cause.

VERIQTA

HTTP 500: Configuration and Filesystem Failures

23 Nginx returns 500 after a configuration change

Incident

Nginx returns HTTP 500 Internal Server Error after a configuration change.

What it means

- Nginx encountered an internal error while processing the request due to a configuration issue.
- This can happen because of rewrite loops, misconfigured error handling, or invalid directives.
- It is a server-side error (5xx), not a client failure.

Common causes

- Internal rewrite loops (e.g. incorrect rewrite or try_files).
- Misconfigured error_page causing a loop.
- Invalid or conflicting directives in the Nginx configuration.
- Incorrect upstream configuration.
- FastCGI/Proxy configuration errors.
- Included configuration files with syntax or logic errors.
- Permissions issues on referenced files or directoires.

Example error log (Nginx)

```
2026/10/27 14:23:11 [error] 12345#12345:
rewrite or internal redirection cycle
while processing "/", client: 203.0.113.10,
server: example.com, request: "GET / HTTP/1.1"

2026/10/27 14:25:02 [error] 12345#12345:
open() "/var/www/html/index.html" failed
(13: Permission denied), client: 203.0.113.10,
server: example.com, request: "GET / HTTP/1.1"
```

How to investigate

1. Test the Nginx configuration:
   `nginx -t`
2. Check recent configuration changes:
   `git diff /etc/nginx/`
3. Review Nginx error logs:
   `journalctl -u nginx -n 100`
   `# or tail -n 100 /var/log/nginx/error.log`
4. Check for rewrite loops, error_page settings and upstream configuration.
5. Verify file and directory permissions.

Fix and verify

- Correct rewrite rules or remove rewrite loops.
- Fix error_page configuration.
- Update invalid or conflicting directives.
- Reload Nginx:
  `nginx -s reload`
- Test the application and confirm the 500 error is resolved.

Example configuration (correct)

```
server {
    listen 80;
    server_name example.com;

    root /var/www/html;
    index index.html index.htm;

    location / {
        try_files $uri $uri/ =404;
    }

    error_page 500 502 503 504
        /50x.html;
    location = /50x.html {
        root /usr/share/nginx/html;
    }
}
```

24 A web application returns 500 when writing uploaded files

Incident

A web application returns HTTP 500 Internal Server Error when writing uploaded files.

What it means

- The application is unable to write the uploaded file to the filesystem.
- This usually happens due to missing directories, insufficient permissions, read-only mounts, or exhausted storage.
- It is a server-side error (5xx), not a client failure.

Common causes

- Missing upload directory.
- Incorrect filesystem permissions.
- Read-only filesystem or container volume.
- Insufficient disk space (disk full).
- Security policies (SELinux/AppArmor).
- Incorrect application configuration for upload path.
- File ownership mismatch (e.g. Nginx user vs application user).

Example error log (application)

```
ERROR 2026-10-27 14:40:21
java.io.IOException: Permission denied
    at java.base/sun.nio.fs.UnixException
    at java.base/sun.nio.fs.UnixFileSystemProvider
    ...
Caused by: java.io.FileNotFoundException:
/var/www/uploads/image.jpg (No such file or directory)
```

Disk usage check

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   50G     0 100% /
/dev/sdb1       200G  120G   80G  60% /data
overlay          20G   20G     0 100% /var/lib/docker
```

How to investigate

1. Check application logs for detailed error messages.
2. Verify the upload directory exists:
   `ls -ld /var/www/uploads`
3. Check directory permissions:
   `ls -l /var/www/uploads`
4. Check disk space:
   `df -h`
5. Verify if the filesystem is read-only:
   `mount | grep "/var/www/uploads"`
6. Test by creating a file manually with the application user.

Fix and verify

- Create the missing directory.
- Set correct permissions and ownership (e.g. chown, chmod).
- Ensure the filesystem has sufficient disk space.
- Remount with read-write access if needed.
- Update application configuration to use the correct upload path.
- Test file upload again and confirm the 500 error is resolved.

Example fix (permissions)

```
sudo mkdir -p /var/www/uploads
sudo chown -R www-data:www-data \
    /var/www/uploads
sudo chmod -R 755 /var/www/uploads
```

Engineering focus Use nginx -t, journalctl, df -h, and targeted filesystem inspection to identify configuration or storage failures.

VERIQTA

HTTP 501 and 505: Unsupported HTTP Capabilities

25 A server returns 501 Not Implemented for an unsupported request method

Incident

A server returns HTTP 501 Not Implemented when an unsupported request method is used.

What it means

- The server does not support the request method.
- The requested functionality is not implemented on the server.
- This happens when a method (e.g. PUT, DELETE, PATCH, etc.) is not supported at all.
- It is a server-side error (5xx), not a client failure.

Common causes

- Server does not implement the requested method (e.g. PUT, PATCH, DELETE).
- Method disabled in web server or application.
- Using a method not supported by the server software or version.
- Proxy or load balancer blocking or removing the method.
- API or application framework not configured to handle the method.

Example request and response

```
curl -i -X PUT https://api.example.com/users/123

HTTP/1.1 501 Not Implemented
Content-Type: application/json

{
  "error": "not_implemented",
  "message": "The PUT method is
    not supported on this server."
}
```

How to investigate

1. Check which methods are allowed:
   `curl -i -X OPTIONS https://api.example.com`
2. Verify server capabilities and documentation for supported methods.
3. Check web server or application configuration (e.g. Nginx, Apache, application framework).
4. Review proxy or load balancer method restrictions.
5. Test with a supported method (e.g. GET or POST).

Fix and verify

- Use a supported HTTP method for the endpoint.
- Enable the required method in the server or application (if it should be supported).
- Update proxy or load balancer configuration to allow the method (if applicable).
- Test again and confirm the server no longer returns 501.

Example: Check allowed methods

```
curl -i -X OPTIONS https://api.example.com

HTTP/1.1 200 OK
Allow: GET, POST, OPTIONS
Content-Type: text/plain
```

Example: Nginx allow specific methods

```
location /api/ {
    limit_except GET POST {
        deny all;
    }
}
```

26 A legacy service returns 505 HTTP Version Not Supported

Incident

A legacy service returns HTTP 505 HTTP Version Not Supported when the client uses an unsupported HTTP version.

What it means

- The server does not support the HTTP version used by the client.
- This usually happens with old or legacy services that only support older HTTP versions (e.g. HTTP/1.0).
- It is a server-side error (5xx), not a client failure.

Common causes

- Client is using a newer HTTP version (e.g. HTTP/2 or HTTP/3) not supported by the server.
- Legacy server only supports HTTP/1.0 or HTTP/1.1.
- Proxy or load balancer is forwarding a different HTTP version to the backend.
- Misconfigured protocol settings in web server, gateway or proxy.
- Incompatible client-server combination.

Example request and response

```
curl --http2 -i https://legacy.example.com

HTTP/2 505 HTTP Version Not Supported
Content-Type: text/html

<html>
  <head><title>505 HTTP Version Not Supported</title></head>
  <body>
    <h1>505 HTTP Version Not Supported</h1>
    <p>The server does not support the HTTP version
      used in the request.</p>
  </body>
</html>
```

How to investigate

1. Check the HTTP version used by the client:
   `curl -I --http2 https://legacy.example.com`
2. Try using a different HTTP version:
   `curl -I --http1.1 https://legacy.example.com`
3. Verify server documentation for supported versions.
4. Check proxy or load balancer protocol settings.
5. Review server logs for version-related errors.

Fix and verify

- Configure the client to use a supported HTTP version (e.g. HTTP/1.1).
- Update proxy or load balancer to use a compatible version with the backend.
- Upgrade the server if it should support newer HTTP versions.
- Test again and confirm the server responds without 505.

Example: Use HTTP/1.1 with curl

```
curl --http1.1 -I https://legacy.example.com

HTTP/1.1 200 OK
Server: LegacyServer/1.0
Content-Type: text/html
```

Example: Nginx proxy protocol version

```
proxy_pass http://legacy-backend;
proxy_http_version 1.1;
proxy_set_header Connection "";
```

Engineering focus Distinguish unsupported server functionality from a known endpoint that rejects a particular method, and verify HTTP version compatibility between client, proxy and server.

VERIQTA

HTTP 502: Bad Gateway After Deployment

27 Nginx returns 502 because the upstream application is unavailable

Incident

Nginx returns HTTP 502 Bad Gateway because the upstream application is unavailable after a deployment.

What it means

- Nginx (reverse proxy) cannot connect to the upstream application.
- The upstream application may be stopped, crashing, or listening on the wrong port.
- This is usually due to connection refusals, incorrect upstream configuration, or application startup failures.
- It is a server-side error (5xx), not a client failure.

Common causes

- Application process is not running or crashed.
- Upstream listening on the wrong IP or port.
- Connection refused by upstream.
- Application failed to start after deployment.
- Incorrect upstream configuration in Nginx (wrong host, port, or path).
- Firewall or security group blocking Nginx to upstream.
- Health check misconfiguration (marking healthy when not ready).
- Dependency services (database, cache) not available.

Example error log (Nginx)

```
2026/10/27 15:21:10 [error] 12345#12345:
*1024 connect() failed (111: Connection
refused) while connecting to upstream,
client: 203.0.113.10, server: example.com,
request: "GET /api/ HTTP/1.1",
upstream: "http://127.0.0.1:8000/api/"",
host: "example.com"
```

Example response (client)

```
curl -i https://example.com/api/
HTTP/1.1 502 Bad Gateway
Server: nginx
Content-Type: text/html
<html>
<head><title>502 Bad Gateway</title></head>
<body>
 <h1>502 Bad Gateway</h1>
</html>
```

How to investigate

1. Check if the application is running: `systemctl status <app>` or `ps -ef | grep <app>`
2. Check if the application is listening on the expected port: `ss -lntp | grep <port>`
3. Test upstream directly from the server: `curl -i http://127.0.0.1:8000/health`
4. Review Nginx error logs: `tail -n 100 /var/log/nginx/error.log`
5. Review application logs: `journalctl -u <app> -n 100`

Fix and verify

- Start or restart the application process.
- Fix the upstream host, IP or port in Nginx configuration.
- Ensure the application is listening and reachable.
- Check firewall or security group rules.
- Fix application startup errors and missing dependencies.
- Reload Nginx after changes: `nginx -s reload`
- Test again and confirm the upstream responds successfully (HTTP 200).

Example: Check upstream connectivity

```
# Check if port is listening
ss -lntp | grep 8000
LISTEN    0    128   0.0.0.0:8000
users:(("python",pid=1234,fd=3))

# Test upstream directly
curl -i http://127.0.0.1:8000/health
HTTP/1.1 200 OK
Content-Type: application/json
{"status":"ok"}
```

28 A reverse proxy intermittently returns 502 due to upstream issues

Incident

A reverse proxy intermittently returns HTTP 502 Bad Gateway due to upstream connection problems.

What it means

- Nginx can connect to the upstream sometimes, but the connection fails intermittently.
- This usually happens due to prematurely closed connections, upstream crashes, stale keepalive connections, or protocol mismatches.
- It is a server-side error (5xx).

Common causes

- Upstream application crashes under load.
- Prematurely closed connections by the upstream.
- Stale keepalive connections.
- Connection timeouts or ressts.
- Protocol mismatch (e.g. HTTP vs HTTPS, HTTP/1.1 vs HTTP/2).
- Application resource exhaustion (CPU, memory, file descriptors).
- Load balancer forwarding to unhealthy instances.
- Network interruptions between Nginx and upstream.

Example error log (intermittent)

```
2026/10/27 16:02:14 [error] 12345#12345:
*2048 upstream prematurely closed connection
while reading response header from upstream,
client: 203.0.113.10, server: example.com,
request: "GET /api/data HTTP/1.1",
upstream: "http://10.0.1.15:8000/api/data",
host: "example.com"
```

How to investigate

1. Check upstream health and response time: `curl -i http://<upstream-host>:<port>/health`
2. Monitor upstream application logs for crashes or errors.
3. Check for high resource usage (CPU, memory, file handles).
4. Verify keepalive and timeout settings in Nginx and upstream.
5. Check for protocol mismatches (HTTP/HTTPS, HTTP/1.1 vs HTTP/2).

Fix and verify

- Improve application stability und resource limits.
- Adjust Nginx proxy settings (proxy_read_timeout, keepalive, health checks).
- Fix protocol mismatches.
- Use proper load balancer health checks.
- Monitor and test under load.
- Confirm the 502 errors no longer occurs.

Example: Check upstream status

```
# Check upstream endpoint
curl -I http://10.0.1.15:8000/api/data
HTTP/1.1 200 OK

# Check multiple times to test stability
for i in {1..5}; do
  curl -s -o /dev/null -w "%{http_code}\n" \
  http://10.0.1.15:8000/api/data
done
```

Engineering focus Use `ss -lntp`, `curl`, Nginx error logs, and service logs to establish whether the upstream is reachable and responding correctly.

VERIQTA

HTTP 502: Kubernetes and Load Balancer Failures

29 Kubernetes Ingress returns 502 after a rollout

Incident

Kubernetes Ingress returns HTTP 502 Bad Gateway after a rollout when accessing the application.

What it means

- The Ingress controller cannot connect to the backend Service or Pod.
- This usually happens due to incorrect Service targetPort, unavailable application listeners, protocol mismatches, or failing Pods.
- It is a server-side error (5xx), not a client failure.

Common causes

- Incorrect Service targetPort or port mapping.
- Application not listening on the expected port.
- Pod not ready or failing health checks.
- Protocol mismatch (e.g. HTTP vs HTTPS).
- Misconfigured Ingress annotations or backend configuration.
- Ingress controller issues or resource limits.
- Network policies blocking traffic between Ingress and Pods.

Example (Kubernetes resources)

Check Service and endpoints:

```
kubectl get svc <service-name> -o wide
kubectl get endpoints <service-name>
```

Check Pod status:

```
kubectl get pods -l app=<app-name>
kubectl describe pod <pod-name>
```

Check Ingress controller logs:

```
kubectl -n ingress-nginx logs \
 -l app.kubernetes.io/name=ingress-nginx \
 --tail=100
```

Example 502 error in Ingress log:

```
2026/10/27 12:14:33 [error] 12345#12345:
*9876 connect() failed (111: Connection
refused) while connecting to upstream,
client: 10.0.0.1, server: example.com,
request: "GET / HTTP/1.1", upstream:
"http://10.1.2.3:8080/".
```

How to investigate

1. Check Ingress, Service and Endpoint configuration:
   `kubectl get ingress,svc,endpoints -o wide`
2. Verify Pod health and logs:
   `kubectl get pods -l app=<app-name>`
   `kubectl logs <pod-name> --tail=100`
3. Check if the application is listening on the correct port:
   `kubectl exec -it <pod-name> -- ss -lntp`
4. Review Ingress controller logs for 502 errors.
5. Verify network policies and security groups.

Fix and verify

- Correct Service targetPort and port mappings.
- Ensure the application is listening on the expected port.
- Verify Pods are ready and passing health checks.
- Fix protocol mismatches (HTTP/HTTPS).
- Update Ingress configuration or annotations.
- Reload or restart the Ingress controller if needed.
- Test again and confirm the 502 error is resolved.

30 A cloud load balancer returns 502 while backend instances appear healthy

Incident

A cloud load balancer returns HTTP 502 Bad Gateway while backend instances appear healthy and are passing health checks.

What it means

- The load balancer could not get a valid response from the backend instances.
- This can happen due to malformed upstream responses, backend connection resets, TLS mismatches, or load balancer-specific issues.
- It is a server-side error (5xx), not a client failure.

Common causes

- Malformed or invalid upstream responses from the backend.
- Backend connection resets or timeouts.
- TLS/SSL mismatches between load balancer and backend.
- Protocol version mismatches (e.g. HTTP/1.1 vs HTTP/2).
- Load balancer idle timeout or keepalive issues.
- Backend becoming unhealthy intermittently under load.
- Load balancer configuration issues or limits.

Example (load balancer logs/metrics)

AWS ALB example (CloudWatch log):

```
2026-10-27T12:45:01Z elb 502 - - - -
"GET /api HTTP/1.1" 502 - "-" "curl/8.0"
"-" "10.0.1.2:8080" "error_reason=\"
invalid_response\""
```

Nginx behind load balancer (upstream error):

```
2026/10/27 12:45:01 [error] 12345#12345:
*4321 upstream prematurely closed connection
while reading response header from upstream,
client: 203.0.113.10, server: example.com,
request: "GET /api HTTP/1.1", upstream:
"http://10.0.2.5:8080/api"
```

How to investigate

1. Check load balancer logs and metrics for 502 details.
2. Verify backend instance health and application logs.
3. Test backend directly (bypass load balancer):
   `curl -i http://<backend-ip>:<port>/api`
4. Check TLS configuration, protocols and certificates.
5. Review timeouts, keepalive settings and connection limits.

Fix and verify

- Fix malformed responses or application errors.
- Ensure stable backend connections.
- Resolve TLS/SSL mismatches and protocol issues.
- Adjust load balancer timeouts and keepalive settings.
- Test end-to-end flow through the load balancer, ingress, Service, endpoint and Pod.
- Confirm the 502 error is resolved.

Example: Test backend directly

```
# Test backend health
curl -i http://10.0.2.5:8080/health

# Test with HTTPS if required
curl -k https://10.0.2.5:8443/health

# Check TLS handshake
openssl s_client -connect \
 10.0.2.5:8443 -servername example.com
```

Engineering focus Trace traffic through the load balancer, ingress, Service, endpoint, and Pod. Verify which component actually generated the 502.

VERIQTA

HTTP 503: Service Unavailable

31 A Kubernetes application returns 503 during a rolling deployment

Incident

A Kubernetes application returns HTTP 503 Service Unavailable during a rolling deployment.

What it means

- The application is temporarily unavailable and cannot handle the request.
- This usually happens when there are no ready endpoints, readiness probes are failing, or there is temporary capacity shortage during a rollout.
- It is a server-side error (5xx), not a client failure.

Common causes

- No ready endpoints in the Service.
- Readiness probe failures.
- Pods not ready during rollout (startup or slow initialization).
- Incorrect readiness probe configuration (wrong path, port, or response code).
- Rolling deployment reducing capacity (maxUnavailable).
- Application failing health checks.
- Resource constraints (CPU, memory).
- Image pull or startup failures.
- Ingress or load balancer sending traffic to unready Pods.

Example: Check pods and status

```
kubectl get pods -o wide
kubectl describe pod <pod-name>
kubectl logs <pod-name> --tail=100

NAME                  READY  STATUS   RESTARTS  AGE
api-7d9f5c6b9-abc12   0/1    Running  3         2m
api-7d9f5c6b9-def34   1/1    Running  0         2m
```

Example: Check endpoints

```
kubectl get endpointslices -l kubernetes.io/
service-name=api-service -o wide

NAME        ADDRESSES   READY   PORTS
api-abcde   10.1.2.3    false   8080
api-fghij   10.1.2.4    false   8080
```

How to investigate

1. Check application rollout status:
   `kubectl rollout status deployment/<name>`
2. Check Pod readiness and health:
   `kubectl get pods -o wide`
   `kubectl describe pod <pod-name>`
3. Check readiness probe configuration:
   `kubectl get edployment <name> -o yaml`
4. Check Service endpoints:
   `kubectl get endpointslices -l kubernetes.io/service-name=<service> -o wide`
5. Review application logs and events:
   `kubectl logs <pod-name> --tail=100`
   `kubectl get events --sort-by=.lastTimestamp`

Fix and verify

- Fix application startup or readiness probe issues.
- Ensure enough ready Pods during rollout (adjust maxUnavailable, maxSurge).
- Verify correct Service targetPort and selectors.
- Resolve resource constraints (CPU, memory).
- Ensure health check endpoints return success (200).
- Redeploy and confirm all Pods are ready and endpoints are available.

Example: Readiness probe

```
readinessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10
  timeoutSeconds: 5
  failureThreshold: 3
```

32 An API returns 503 during traffic overload

Incident

An API returns HTTP 503 Service Unavailable during traffic overload.

What it means

- The server is temporarily unable to handle the request due to overload or capacity limits.
- This can happen when workers are saturated, connection limits are reached, or downstream dependencies are unavailable.
- It is a server-side error (5xx), not a client failure.

Common causes

- Worker or thread pool exhaustion.
- High CPU or memory usage.
- Connection limits reached (max_connections).
- Load shedding or circuit breaker activated.
- Database or downstream service unavailable.
- Application queue saturation.
- Auto scaling not keeping up with traffic.
- Rate limiting or throttling at the gateway or load balancer.

Example: Application logs under load

```
2026-10-27 14:21:03 ERROR WorkerPool
- RejectedExecutionException: Task queue full
2026-10-27 14:21:03 WARN RateLimiter
- Request rejected, too many requests
2026-10-27 14:21:04 ERROR Database
- Connection pool exhausted (HikariPool)
```

How to investigate

1. Check application metrics (CPU, memory, request rate, latency).
2. Review application logs for overload or rejection messages.
3. Check downstream dependency health (database, external APIs).
4. Verify connection and worker limits in configuration.
5. Check load balancer or gateway metrics for throttling/load shedding.

Fix and verify

- Increase capacity (scale up or enable auto scaling).
- Optimize worker and connection pool settings.
- Fix slow or failing downstream dependencies.
- Implement load shedding or queueing if needed.
- Monitor metrics and confirm

Example: Check system metrics

```
# Check CPU and memory
kubectl top pods

# Check application metrics (Prometheus)
curl -s http://<app>/metrics

# Check load balancer metrics (AWS example)
aws elbv2 describe-target-health \
  --target-group-arn <tg-arn>
```

Engineering focus Use kubectl get pods, kubectl get endpointslices, kubectl describe, and workload metrics to identify unavailable capacity and confirm when the service is healthy again.

VERIQTA

HTTP 504: Gateway Timeout

33 Nginx returns 504 while an upstream API processes a slow request

Incident

Nginx returns HTTP 504 Gateway Timeout while an upstream API processes a slow request.

What it means

- Nginx (reverse proxy) did not receive a response from the upstream server within the timeout period.
- This usually happens when the upstream API is slow or blocked.
- It can be caused by long-running database queries, external API calls, or resource exhaustion.
- It is a server-side error (5xx), not a client failure.

Common causes

- Slow application processing (e.g. heavy database queries).
- Upstream response latency exceeds proxy_read_timeout.
- External dependency delays (third-party APIs, payment gateways, etc.).
- Incorrect or low timeout settings in Nginx.
- Resource exhaustion (CPU, memory, threads).
- Network latency between Nginx and upstream.
- Application deadlocks or blocked threads.

Example error log (Nginx)

```
2026/10/27 15:24:11 [error] 12345#12345:
*9876 upstream timed out (110: Connection
timed out) while reading response header
from upstream, client: 203.0.113.10,
server: example.com, request: "GET /api/data
HTTP/1.1", upstream: "http://10.0.1.20:8080/
api/data", host: "example.com"
```

Example response (client)

```
curl -i https://example.com/api/data
HTTP/1.1 504 Gateway Timeout
Server: nginx
Content-Type: text/html
<html>
<head><title>504 Gateway Timeout</title></head>
<body>
<h1>504 Gateway Timeout</h1>
</body>
</html>
```

How to investigate

1. Check Nginx error logs:
   `tail -n 100 /var/log/nginx/error.log`
2. Check upstream application logs for slow requests.
3. Check upstream response time directly:
   `curl -v http://<upstream-ip>:<port>/api/data`
4. Review database slow query logs (if applicable).
5. Check proxy timeout settings in Nginx configuration.

Fix and verify

- Identify and fix the underlying slow request (e.g. optimize database queries).
- Ensure upstream application is healthy and responsive.
- Review and adjust timeout settings only after confirming the expected response time.
- Fix external dependency delays or failures.
- Monitor application and database performance.

Example Nginx timeout configuration)

```
location /api/ {
    proxy_pass http://10.0.1.20:8080;
    proxy_connect_timeout 10s;
    proxy_send_timeout 30s;
    proxy_read_timeout 30s;
    send_timeout 30s;
}
```

34 A load balancer returns 504 during network or backend stalls

Incident

A load balancer returns HTTP 504 Gateway Timeout during intermittent network or backend stalls.

What it means

- The load balancer did not receive a response from the backend within its timeout period.
- This can happen due to network issues, stalled responses, idle timeouts, or backend saturation.
- It is a server-side error (5xx), not a client failure.

Common causes

- Upstream application taking too long to respond.
- Idle timeout configuration on the load balancer.
- Backend connection stalls or connection resets.
- Packet loss or high network latency.
- Upstream server saturation (CPU, memory, connection limits).
- TLS handshake delays or misconfigurations.
- Resource constraints or thread exhaustion on backend instances.

Example load balancer log (AWS ALB)

```
2026-10-27T15:40:21.123Z elb 504 - - -
"GET /api/report HTTP/1.1" 504 - "-"
"curl/8.0" "10.0.2.15:8080" 30.001
"error_reason"="target_response_timeout"
"target_status_code"="-"
"target_ip"="10.0.2.15"
"request_processing_time"="0.001"
"target_processing_time"="30.001"
"response_processing_time"="0.000"
```

How to investigate

1. Check load balancer logs for 504 details and target latency.
2. Verify backend instance health and response time:
   `curl -v http://<backend-ip>:<port>/api/report`
3. Check network connectivity and latency (ping, traceroute, mtr).
4. Review backend application logs for slow responses or stalls.
5. Check system metrics (CPU, memory, connection count, thread pool).

Fix and verify

- Identify and fix slow backend responses.
- Resolve network issues or packet loss.
- Adjust idle timeout settings appropriately.
- Increase backend capacity or optimize resource usage.
- Fix connection limits or thread pool exhaustion.
- Monitor metrics and confirm 504 errors are resolved.

Example: Check backend connectivity

```
# Test connectivity to backend
curl -v http://10.0.2.15:8080/api/report

# Check network latency
mtr -rwz -c 10 10.0.2.15

# Check open connections on backend
ss -s

# Check backend application logs
tail -n 100 /var/log/app/app.log
```

Engineering focus Compare proxy and load balancer timeout settings with observed upstream latency. Do not increase timeouts without investigating and resolving the underlying delay.

VERIQTA

HTTP 507 and 508: Storage and Processing Failures

35 A WebDAV-compatible storage service returns 507 Insufficient Storage

Incident

A WebDAV-compatible storage service returns HTTP 507 Insufficient Storage.

What it means

- The server cannot store the requested representation due to insufficient storage space.
- This usually happens when disk space, storage quota, or inode limits are exhausted.
- It can also be caused by failing write operations on the underlying filesystem or storage backend.
- It is a server-side error (5xx), not a client failure.

Common causes

- Disk space full on the server or volume.
- User or tenant storage quota exceeded.
- Inode limit reached (no more files can be created).
- Filesystem read-only or write failures.
- Storage backend unavailable or saturated (e.g. NFS, EFS, S3 gateway, Ceph).
- Large file uploads exceeding allowed limits.
- Temporary storage exhaustion (e.g. in /tmp).

Example error response (WebDAV)

```
HTTP/1.1 507 Insufficient Storage
Content-Type: application/xml

<D:error xmlns:D="DAV:">
  <D:insufficient-storage/>
  <D:message>Not enough space
   to store the resource.</D:message>
</D:error>
```

Example server log

```
2026/10/27 16:12:45 [error] 12345#12345:
*9876 write() "/data/uploads/file.zip"
failed (28: No space left on device)
while handling WebDAV PUT request
```

How to investigate

1. Check disk space and inode usage:
   `df -h` `df -i`
2. Check storage quotas (if applicable):
   `quota -u <user>`
3. Verify filesystem mount options:
   `mount | grep <mount-point>`
4. Check WebDAV or application logs for failed writes:
   `tail -n 100 /var/log/<app>/error.log`
5. Test write operation directly:
   `touch /path/to/testfile`
   `echo "test" > /path/to/testfile`

Fix and verify

- Free up disk space or increase storage capacity.
- Increase user or tenant quotas.
- Clean up old files or logs.
- Restore write permissions.
- Fix underlying storage backend issues.
- Verify the server can write files successfully.
- Retry the WebDAV operation after resolving storage constraints.

Example: Check storage usage

```
# Check disk space
df -h
Filesystem    Size  Used Avail Use%
/dev/sda1     100G   98G  1.2G  99%

# Check inode usage
df -i
Filesystem   Inodes  IUsed IFree IUse%
/dev/sda1    655360 655360     0  100%

# Check mount options
mount | grep /data
/dev/sda1 on /data type ext4 (rw,relatime)
```

36 A server returns 508 Loop Detected during resource processing

Incident

A server returns HTTP 508 Loop Detected during resource processing.

What it means

- The server detected an infinite loop while processing a request.
- This commonly occurs with cyclic resource references or recursive operations, especially in WebDAV-style resource processing.
- It is a server-side error (5xx), not a client failure.

Common causes

- Cyclic links or circular references between resources (e.g. WebDAV collections).
- Infinite recursion in application logic.
- Misconfigured rewrite or internal routing logic that creates a loop (when specifically detected as 508).
- Cyclic dependencies in file synchronization or replication.
- Directory or symlink loops.
- Recursive macro or template processing.

Example error response (WebDAV)

```
HTTP/1.1 508 Loop Detected
Content-Type: application/xml

<D:error xmlns:D="DAV:">
  <D:loop-detected/>
  <D:message>Cycle in resource
   hierarchy detected.</D:message>
</D:error>
```

Example server log

```
2026/10/27 17:03:12 [error] 12345#12345:
*6543 WebDAV: loop detected while
processing resource "/docs/reports"
(client: 203.0.113.10)
```

How to investigate

1. Check the resource structure for circular references:
   `find /path/to/collection -xtype l -exec ls -l {} +`
2. Review WebDAV or application logs for loop details:
   `tail -n 100 /var/log/<app>/error.log`
3. Check for symbolic link loops:
   `ls -lR /path/to/collection`
4. Verify rewrite or routing rules (if applicable). Reproduce the request and trace the resource path.

Fix and verify

- Remove or correct cyclic resource references.
- Fix recursive logic or misconfigured rewrite rules.
- Limit recursion depth in application logic.
- Validate WebDAV collection structure.
- Test the request again and confirm the 508 error is resolved.

Example: Check for symlink loops

```
# Find symbolic links
find /data -xtype l -exec ls -l {} +

# Check a specific path
ls -lR /data/docs

# Check application or WebDAV logs
tail -n 100 /var/log/<app>/error.log
```

Engineering focus Distinguish storage capacity failures from logical processing loops. Recognize that 508 is a specialized status and is not the standard response for every redirect or rewrite loop.

VERIQTA

HTTP 500 and 503: Database and Dependency Incidents

37 An API returns 500 when its database becomes unavailable.

Incident

An API returns HTTP 500 when its database becomes unavailable.

What it means

- The application cannot complete the request due to a database failure.
- This usually happens when the database is unreachable, connections are exhausted, queries fail, or exceptions are not handled properly.
- It is a server-side error (5xx), not a client failure.

Common causes

- Database server is down or unreachable.
- Connection pool exhaustion (max connections reached).
- Slow or failing queries (timeout, deadlock, lock contention).
- Incorrect database credentials or network issues.
- Database resource limits (CPU, memory, storage).
- Unhandled exceptions in the application.

Example application log

```
2026-10-27 10:15:42 [ERROR]
com.example.dao.UserDao
- Failed to execute query
java.sql.SQLExqeption: Connection
refused: could not connect to server
at "10.0.1.50", port 5432
Caused by: org.postgresql.util.PSQLExqeption:
Connection refused
```

Example error response (client)

```
HTTP/1.1 500 Internal Server Error
Content-Type: application/json
{
  "error": "Internal Server Error",
  "message": "Database connection failed"
}
```

How to investigate

1. Check database connectivity:
   `nc -zv <db-host> <db-port>`
2. Check application logs for database errors:
   `tail -n 100 /var/log/app/app.log`
3. Check database status and resources:
   `systemctl status postgresql  # or mysql`
   `df -h  # check disk space`
   `free -h  # check memory`
4. Check connection pool metrics (if available).
5. Verify if queries are failing or timing out.

Fix and verify

- Restore database connectivity and ensure the database is healthy.
- Increase connection pool size if needed.
- Optimize slow queries.
- Fix network, credentials, or firewall issues.
- Implement proper exception handling and return meaningful responses.
- Monitor database metrics and application health.

Example: Check database connectivity

```
# Test connection to PostgreSQL
nc -zv 10.0.1.50 5432

# Test with psql
psql -h 10.0.1.50 -U <user> -d <db>

# Check database status
systemctl status postgresql

# Check active connections
psql -h 10.0.1.50 -U <user> -d <db>
-c "SELECT count(*) FROM pg_stat_activity;"
```

38 A service returns 503 when a critical downstream dependency becomes unavailable.

Incident

A service returns HTTP 503 when a critical downstream dependency becomes unavailable.

What it means

- The service is temporarily unable to handle the request due to an unavailable downstream dependency.
- This can happen when an external API, microservice, or third-party service is down, slow, or failing.
- It is a server-side error (5xx), not a client failure.

Common causes

- Downstream service is down or unreachable.
- High latency or timeouts from the dependency.
- Circuit breaker is open.
- Connection pool or thread exhaustion when calling downstream service.
- Rate limiting or throttling by the downstream service.
- Dependency returning errors (5xx, timeouts).
- Network issues between services (DNS, firewall, load balancer).

Example service log

```
2026-10-27 11:22:10 [WARN] DownstreamService
- GET https://api.payment-provider.com/charge
failed: Read timed out (after 3000 ms)
2026-10-27 11:22:10 [ERROR] CircuitBreaker
- Dependency api.payment-provider.com is OPEN
2026-10-27 11:22:10 [INFO] Returning 503
Service Unavailable due to downstream failure
```

Example error response (client)

```
HTTP/1.1 503 Service Unavailable
Content-Type: application/json
{
  "error": "Service Unavailable",
  "message": "Downstream dependency is
} currently unavailable"
```

How to investigate

1. Check downstream dependency health and status.
2. Review application logs for timeout or connection errors.
3. Check circuit breaker and retry configuration.
4. Verify network connectivity and DNS resolution.
5. Check metrics (latency, error rate, saturation).

Fix and verify

- Restore or fix the downstream service.
- Adjust timeout and retry settings.
- Implement circuit breaker and graceful degradation.
- Verify network and DNS configuration.
- Monitor dependency health and alerts.
- Test the service end-to-end to ensure 503 is resolved.

Example: Check downstream service

```
# Test connectivity and response time
curl -v https://api.payment-provider.com/health
# Check DNS resolution
dig api.payment-provider.com
# Check network path
traceroute api.payment-provider.com
```

Engineering focus Understand that an application chooses how to map dependency failures to HTTP status codes. A database outage does not automatically produce one specific HTTP status.

VERIQTA

HTTP 502 and 504: Multi-Layer Production Incident

39 Users receive intermittent 502 errors after a Kubernetes deployment.

Incident

Users receive intermittent HTTP 502 errors after a Kubernetes deployment.

What it means

- The reverse proxy or ingress cannot successfully connect to the upstream application.
- This usually happens due to application crashes, connection resets, or misconfigured routing after a deployment.
- It is a server-side error (5xx), not a client failure.

Common causes

- Application crashes or failing startup.
- Upstream connection resets.
- Pods not ready or failing readiness probes.
- Incorrect Service targetPort or selector.
- Ingress controller misconfiguration.
- Rolling deployment causing temporary unavailability.
- Resource limits (CPU, memory) causing restarts.
- Network or DNS issues between ingress and Pods.

Example: Check pod status

```
kubectl get pods -o wide

NAME                READY  STATUS            RESTARTS  AGE
api-7d9f5c6b9-abc12 0/1    CrashLoopBackOff  5         3m
api-7d9f5c6b9-def34 1/1    Running           0         3m
api-7d9f5c6b9-ghi56 1/1    Running           0         3m
```

Example: Nginx ingress error log

```
2026/10/27 12:14:21 [error] 12345#12345:
*9876 connect() failed (111: Connection
refused) while connecting to upstream,
client: 203.0.113.10, server: api.example.com,
request: "GET /api/users HTTP/1.1",
upstream: "http://10.1.2.34:8080/api/users"
```

How to investigate

1. Check pod status and recent events:
   `kubectl get pods -o wide`
   `kubectl describe pod <pod-name>`
2. Check deployment rollout status:
   `kubectl rollout status deployment/<name>`
3. Check service and endpoints:
   `kubectl get svc`
   `kubectl get endpointslices`
4. Check ingress controller logs:
   `kubectl logs -n ingress-nginx <pod-name>`
5. Check application logs for crashes or errors:
   `kubectl logs <pod-name> --tail=100`

Fix and verify

- Fix application crashes and ensure successful startup.
- Verify readiness and liveness probes.
- Confirm correct Service selectors and targetPort.
- Roll back or redeploy if needed.
- Ensure sufficient resource limits (CPU, memory).
- Check ingress configuration and upstream service health.
- Verify recovery: no more 502 errors, all Pods ready, stable traffic.

Example: Check service and endpoints

```
kubectl get svc api-service

NAME         TYPE       CLUSTER-IP   PORT(S)
api-service  ClusterIP  10.1.2.3     8080/TCP

kubectl get endpointslices -l kubernetes.io/
service-name=api-service

NAME      ADDRESSES    READY  PORTS
api-abcd  10.1.2.34    true   8080
api-efgh  10.1.2.35    true   8080
```

40 Users receive 504 errors even though Kubernetes Pods report Ready.

Incident

Users receive HTTP 504 errors even though Kubernetes Pods report Ready.

What it means

- The upstream application takes too long to respond.
- Pods may be ready but the application is slow due to database queries, thread exhaustion, or downstream dependency latency.
- It is a server-side error (5xx), not a client failure.

Common causes

- Slow database queries.
- Application thread or worker pool exhaustion.
- Downstream dependency latency or timeouts.
- Gateway or proxy timeout settings (Nginx, ALB, etc.).
- External service slowness (third-party APIs, payment gateways).
- High resource usage (CPU, memory, I/O) causing slow responses.
- Large or complex requests (e.g. reports, exports).

Example: Application log (slow query)

```
2026-10-27 13:22:10 WARN [http-nio-8080]
c.e.service.UserService - Query took 32.5s
SELECT * FROM orders WHERE user_id = 123;
2026-10-27 13:22:10 ERROR Database timeout
org.postgresql.util.PSQLēqepriate:
The connection attempt timed out.
```

Example: Nginx 504 log

```
2026/10/27 13:22:11 [error] 12345#12345:
*6543 upstream timed out (110: Connection
timed out) while reading response header
from upstream, client: 203.0.113.10,
server: api.example.com, request: "GET /api/
report HTTP/1.1", upstream: "http://10.1.2.34:
8080/api/report", host: "api.example.com"
```

How to investigate

1. Check application logs for slow requests and errors.
2. Review database query logs and performance metrics.
3. Check application metrics (thread pool, latency, response time).
4. Verify downstream dependencies and their response times.
5. Check proxy/gateway timeout settings and compare with actual latency.

Fix and verify

- Optimize slow database queries (add indexes, rewrite queries).
- Increase worker(thraad capacity if needed.
- Fix downstream dependency issues.
- Adjust timeout settings appropriately (after mvestigation).
- Monitor metrics and confirm response times are within expected limits.
- Verify recovery: no more 504 errors, stable latency and traffic.

Example: Check latency metrics

```
# Check application metrics (Prometheus)
curl -s http://<app>/metrics
# Check database slow queries
tail -n 100 /var/log/postgresql/slow.log
# Check upstream response time
curl -w "time_total: %{time_total}\n" \
     http://10.1.2.34:8080/api/health
```

Engineering focus Conduct an end-to-end incident investigation using request IDs, proxy logs, application logs, latency metrics, Kubernetes state, and recent changes. Finish by verifying recovery and identifying preenttive actions.